

하이브리드 클라우드 DR 및 네트워크 이중화

2023733044 박혜소

시스템 구성 및 토폴로지

온프레미스 구축

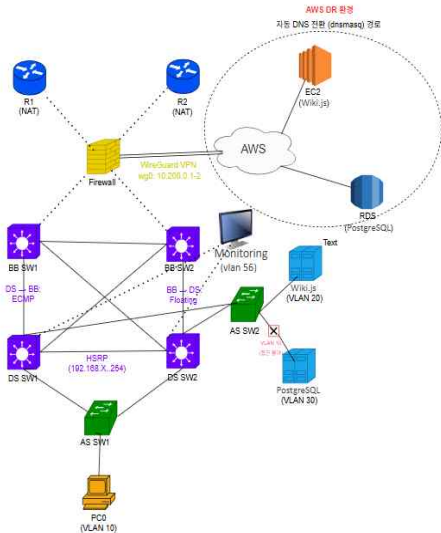
- VLAN 기반 내부망 구성 및 이중화
- Wiki.js / PostgreSQL 서비스 운영

AWS DR 서비스

- 장애 감지 시 AWS DR 환경으로 자동 전환
- DNS 기반 Failover 기반 서비스 연속성 보장

Monitoring Server

- 서비스 및 네트워크 장애 상태 감지
- 자동 DNS 전환 스크립트 실행



OSPF 기반 네트워크 이중화 및 경로 우회 구현

[이론]

(1) OSPF(Open Shortest Path First):

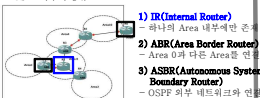
최단 경로를 계산하는 링크 상태 기반 라우팅 프로토콜

링크 상태 기반으로 동작하는 라우팅 프로토콜
Cost 기반 SPF 알고리즘으로 최단 경로 계산
복원 우월 및 부분 업데이트 지원

OSPF 토폴로지 유형



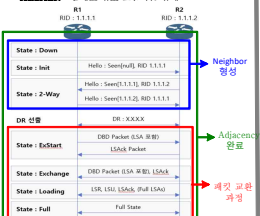
OSPF 라우터 종류



이웃 (neighbor) 형성 과정

- Neighbor: 서로 인식한 상태

- Adjacency: 상세로 정보 교환하는 상태



OSPF 캐스트 주소 (멀티캐스트)

- 224.0.0.5 : 모든 OSPF 라우터 브로드캐스트 사용 X
- 224.0.0.6 : DR / BDR 모든 장비에게 전송 -> 낭비

ospf 설정 명령어

```
R1(config)#router ospf [process-ID]
R1(config-router)#router-id x.x.x.x
R1(config-router)#network x.x.x.x y.y.y.y area 0
```

show ip ospf neighbor

Neighbor ID	Pri	State	Dead Time	Address	Interface
192.168.2.2	1	2WAY/DROTHER	00:00:37	10.121.4.2	SigabitEthernet0/1
192.168.3.3	1	2WAY	00:00:35	10.123.4.3	SigabitEthernet0/1
192.168.4.4	1	2WAY	00:00:33	10.123.4.4	SigabitEthernet0/1

Cost(비용) 기반 경로 계산

Cost = 기준 대역폭 (Reference Bandwidth)
인터페이스의 실제 대역폭 (Interface Bandwidth)

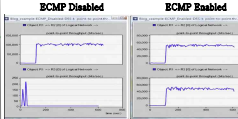
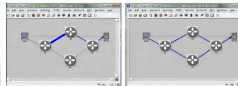
기준 대역폭은: 100 Mbps

인터페이스 실제 대역폭: 하드웨어적으로 정해진 속도

(2) ECMP(Equal-Cost Multi-Path)

동일한 비용(Cost)을 가지는 여러 개의 경로들을 동시에 사용

S*	0.0.0.0/0 [110/0] via 192.168.1.99, port1
S	10.0.4.0/24 [110/0] via 172.16.23.254, port4, [5/0]
	[110/0] via 172.16.39.254, port3, [5/0]
O	10.10.19.0/24 [110/101] via 192.168.2.110, port1, 02:40:45
O	10.10.100.0/24 [110/101] via 192.168.1.110, port1, 02:40:45
C	172.16.20.0/24 is directly connected, port4



ECMP 적용 시 트래픽 분산과 성능 향상

[실습]

이벤트 실험: OSPF 기반 경로 이중화 및 우회



1) L3 인터페이스 구성 및 ping 테스트

```
BB-SW1(config)#int fa0/23
BB-SW1(config-if)#no shutdown
BB-SW1(config-if)#ip address 10.0.1.9 0.0.0.255 area 0
BB-SW1(config-if)#no shutdown
BB-SW1(config-if)#ip address 10.0.1.17 255.255.255.252
BB-SW1(config-if)#no shutdown
```

L3 인터페이스 구성

통신검증(Ping 테스트)

2) OSPF 설정 (Neighbor 관계 형성)

```
ip routing
router ospf 1
router-id 10.0.1.9
network 10.0.1.9 0.0.0.255 area 0
```

OSPF 프로세스

```
BB-SW1#show ip ospf neighbor
```

Neighbor ID	Pri	State	Dead Time	Address
192.168.40.2	1	FULL BDR	00:00:32	10.0.1.6
192.168.40.1	1	FULL BDR	00:00:31	10.0.1.2
10.0.1.18	1	FULL BDR	00:00:38	10.0.1.18

OSPF Neighbor 상태 확인

3) loopback(루프백) 생성

```
BB-SW1(config)#router ospf 1
BB-SW1(config-router)#router-id 1.1.1.1
BB-SW1(config-router)#network 1.1.1.1 0.0.0.0 area 0
```

4) 라우팅 테이블

```
Gateway of last resort is not set
1.1.1.0/32 is subnetted, 1 subnets
C 1.1.1.1 is directly connected, Loopback0
C 10.0.1.0/24 is subnetted, 2 subnets
C 10.0.1.0 is directly connected, FastEthernet0/23
C 10.0.1.4 is directly connected, FastEthernet0/24
O 10.0.1.8 [110/2] via 10.0.1.4, 05:40:27, FastEthernet0/23
O 10.0.1.12 [110/2] via 10.0.1.4, 05:40:27, FastEthernet0/23
```

5) OSPF 내부 확장 (중요)

```
DS-SW1(config)#router ospf 1
DS-SW1(config-router)#network 192.168.10.0 0.0.0.255 area 0
DS-SW1(config-router)#network 192.168.20.0 0.0.0.255 area 0
DS-SW1(config-router)#network 192.168.30.0 0.0.0.255 area 1
DS-SW1(config-router)#network 192.168.40.0 0.0.0.255 area 1
```

VLAN 네트워크 OSPF 광고

```
DS-SW1(config-router)#passive-interface vlan 10
DS-SW1(config-router)#passive-interface vlan 20
DS-SW1(config-router)#passive-interface vlan 30
DS-SW1(config-router)#passive-interface vlan 40
```

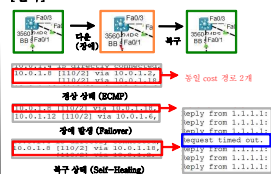
OSPF Hello 차단 (Passive 설정)

```
DS-SW1#show ip ospf neighbor
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
```

5) hello/dead interval 튜닝

```
DS-SW1#show ip ospf neighbor
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
Neighbor 1.1.1.1 is not up
Neighbor 1.1.1.1 with 32 bytes of data
```

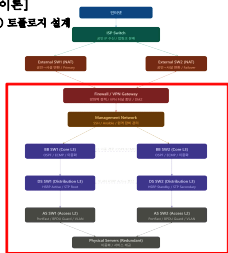
[결과]



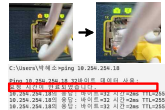
Active/Active 이중화 설정 및 네트워크 구조의 실장비 구현 및 검증

[이론]

1) 토폴로지 설계



2) PortFast 설정



3) BPDUGuard 설정

-> BPDU 들어오면 포트 꺼버리는 보안 기능



- PortFast 포트 보호 (비정상 BPDU 차단)
- BPDU 수신 시 포트 자동 shutdown (err-disable)
- 사용자 단말 포트에만 적용 (보안 기능)

[실험]

1) 토폴로지 설계 및 구축



4) 사용자 PC 연결 (vian 10)



2) 네트워크 기본 설정

VLAN	VLAN Name	Switch-2#sh ip int brief
1	default	IP-Address unassigned
10	User	Vlan10 192.168.10.1
20	Server	Vlan20 192.168.20.1
30	Security	Vlan30 192.168.30.1
99	MANAGEMENT	



3) 사용자 PC 설정

PC	이름	IP 주소	Subnet	Gateway
1	PC1	192.168.10.10	255.255.255.0	192.168.10.1
2	PC2	192.168.20.1	255.255.255.0	192.168.20.1

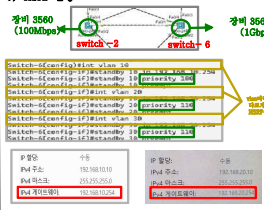
3) access/trunk 설정

Port	Mode	Encapsulation
Gi0/3	active	802.1q
Gi0/4	active	802.1q
Port	Vlans allowed on trunk	
Gi0/3	10, 20	
Gi0/4	10, 20	

5) 1차 ping 테스트

PC: 192.168.10.1 (vian 10)	PC: 192.168.20.1 (vian 20)
Ping 192.168.20.1 32바이트 데이터 사용: 192.168.10.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255 192.168.10.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255 192.168.10.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255 192.168.10.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255	Ping 192.168.10.1 32바이트 데이터 사용: 192.168.20.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255 192.168.20.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255 192.168.20.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255 192.168.20.1: 0% 패킷 손실, 0ms RTT, 0ms TTL=255

6) HSRP 설정



7) HSRP 설정 결과

Switch-2#sh standby brief	Switch-6#sh standby brief
P indicates configured to preempt.	P indicates configured to preempt.
Interface Grp Pri P State Active Standby	Interface Grp Pri P State Active Standby
Vl10 10 110 P Active local 192.168.10.2	Vl10 10 110 P Active local 192.168.10.2
Vl20 20 110 P Active local 192.168.20.2	Vl20 20 110 P Active local 192.168.20.2
Vl30 30 110 P Active local 192.168.30.2	Vl30 30 110 P Active local 192.168.30.2

8) 2차 ping 테스트

Switch-2#sh standby brief	Switch-6#sh standby brief
P indicates configured to preempt.	P indicates configured to preempt.
Interface Grp Pri P State Active Standby	Interface Grp Pri P State Active Standby
Vl10 10 110 P Active local 192.168.10.2	Vl10 10 110 P Active local 192.168.10.2
Vl20 20 110 P Active local 192.168.20.2	Vl20 20 110 P Active local 192.168.20.2
Vl30 30 110 P Active local 192.168.30.2	Vl30 30 110 P Active local 192.168.30.2

9) STP 튜닝

Switch-2#sh spanning-tree vian 10 priority 4096	Switch-6#sh spanning-tree vian 10 priority 4096
Switch-2#sh spanning-tree vian 10 priority 4096	Switch-6#sh spanning-tree vian 10 priority 4096
Switch-2#sh spanning-tree vian 10 priority 4096	Switch-6#sh spanning-tree vian 10 priority 4096
Switch-2#sh spanning-tree vian 10 priority 4096	Switch-6#sh spanning-tree vian 10 priority 4096

10) rapid-pvst+ 설정

Switch-2#sh spanning-tree summary	Switch-6#sh spanning-tree summary
Switch-2#sh spanning-tree summary	Switch-6#sh spanning-tree summary
Switch-2#sh spanning-tree summary	Switch-6#sh spanning-tree summary
Switch-2#sh spanning-tree summary	Switch-6#sh spanning-tree summary

11) portfast & BPDUGuard 설정

Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10
Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10
Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10
Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10

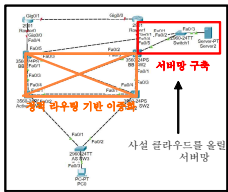
[결과 -1] vian 10 다름 (동시에 vian 10, vian 20에서 ping)

Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10
Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10
Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10
Switch-2#sh spanning-tree vian 10	Switch-6#sh spanning-tree vian 10

리스닝과 러닝 시간 생략 -> 타이머 아웃 시간 줄어듦

클라우드 서버망 & 코어 이중화 구성 및 최종 점검

[실험]



1) 장비 구축 (서버 스위치 + 플리 서버)

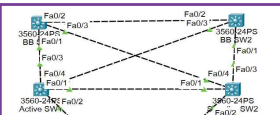


-서버 연결-

2) L3 인터페이스 구성

```
Switch-4(config)#int gi0/1
Switch-4(config-if)#no shutdown
Switch-4(config-if)#ip add 10.0.1.13 255.255.255.252
Switch-4(config-if)#no shut
```

L3 인터페이스 구성



3) 이중화 경계 라우팅 설정 (floating & ECMP)

- DS SW -> BB SW 방향 (floating) -

```
Switch-2(config)#ip route 0.0.0.0/0 10.0.1.1
Switch-2(config)#ip route 0.0.0.0/0 10.0.1.9
```

메인 경로 1개 S* 0.0.0.0/0 [1/0] via 10.0.1.1

```
Switch-6(config)#ip route 0.0.0.0/0 10.0.1.13
Switch-6(config)#ip route 0.0.0.0/0 10.0.1.5
```

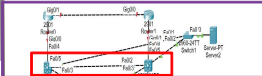
메인 경로 1개 S* 0.0.0.0/0 [1/0] via 10.0.1.13

- BB SW -> DS SW 방향

```
Switch-5(config)#ip route 192.168.30.0/24 10.0.1.10
Switch-5(config)#ip route 192.168.30.0/24 10.0.1.14
Switch-5(config)#ip route 192.168.20.0/24 10.0.1.10
Switch-5(config)#ip route 192.168.20.0/24 10.0.1.14
Switch-5(config)#ip route 192.168.30.0/24 10.0.1.10
Switch-5(config)#ip route 192.168.30.0/24 10.0.1.14
```

경로 2개 (AD값 동일)

```
S 192.168.30.0/24 [1/0] via 10.0.1.14
S 192.168.30.0/24 [1/0] via 10.0.1.10
S 192.168.20.0/24 [1/0] via 10.0.1.14
S 192.168.20.0/24 [1/0] via 10.0.1.10
```



4) BB SW HSRP 게이트웨이 이중화

-BB SW1 (Active)-

```
Switch-4(config)#int vlan 50
Switch-4(config-if)#ip add 172.16.50.1 255.255.255.0
Switch-4(config-if)#standby 50 ip 172.16.50.254
Switch-4(config-if)#standby 50 priority 110
Switch-4(config-if)#standby preempt
```

-BB SW2 (standby)-

```
Switch-5(config)#int vlan 50
Switch-5(config-if)#ip add 172.16.50.2 255.255.255.0
Switch-5(config-if)#standby 50 ip 172.16.50.254
```

Interface	Grp	Pri	State	Active	Standby	Virtual IP
Vlan50	50	110	Active	local	172.16.50.2	172.16.50.254

-> BB SW1

4) 클라우드 서버망 구축

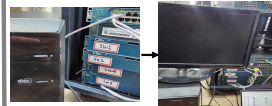
-BB SW's -> trunk

Port	Native vlan	Mode	Encapsulation	Status
Gi0/4/1	1	on	802.1q	trunking

```
Switch-5#sh int trunk
Port Native vlan Mode Encapsulation Status
Gi0/4 1 on 802.1q trunking
```

-server SW -> trunk 및 access

```
Switch(config)#int fa0/1
Switch(config-if)#switchport mode trunk
Switch(config-if)#int fa0/2
Switch(config-if)#switchport mode trunk
```



IP 설정	수동	C-Numbers
수동	172.16.50.13	172.16.50.254
수동	172.16.50.2	172.16.50.254
수동	172.16.50.2	172.16.50.254
수동	172.16.50.2	172.16.50.254

5) 서버망 경로 DS SW에 추가 설정

DS SW1 -> 서버 경로 추가

```
Switch-2(config)#ip route 172.16.50.0 255.255.255.0 10.0.1.1
Switch-2(config)#
```

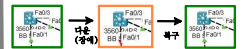
172.16.0.0/24 is subnetted, 1 subnets
S 172.16.50.0 [1/0] via 10.0.1.1

DS SW2 -> 서버 경로 추가

```
Switch-6(config)#ip route 172.16.50.0 255.255.255.0 10.0.1.13
Switch-6(config)#
```

172.16.0.0/24 is subnetted, 1 subnets
S 172.16.50.0 [1/0] via 10.0.1.13

사용자 PC -> 서버 ping 실행..



1) 경계 라우팅 설정 결과

S* 0.0.0.0/0 [1/0] via 10.0.1.1

Switch-4(config)#int gi0/1

Switch-4(config-if)#no shut

S* 0.0.0.0/0 [1/0] via 10.0.1.9

Switch-2#

2) BB SW HSRP 설정 결과

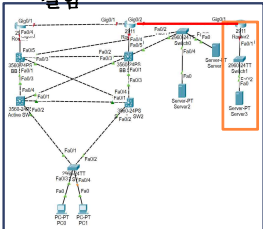
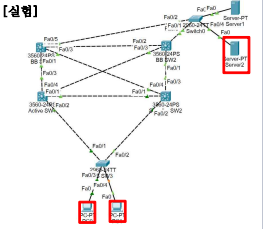
```
Switch-4(config)#int vlan50
Switch-4(config-if)#shut
```

- BB SW2 결과 창 -

Interface	Grp	Pri	State	Active	Standby
Vlan50	50	100	Active	local	BB SW unknown

정책 기반 네트워크 장애 감지 및 자동 경로 우회 실험

[실험]



5. OSPF 적용 (기존 OSPF 1에 광고)

사용자 PC -> 서버방 연결

```
BB_SW1(config)#router ospf 1
BB_SW1(config-router)# network 172.16.20.0 0.0.0.255 area 0
BB_SW1(config-router)# network 172.16.40.0 0.0.0.255 area 0
```

```
0 172.16.20.0 (110/2) via 10.0.1.1, 00:00:52, FastEthernet0/3
0 172.16.40.0 (110/2) via 10.0.1.1, 00:00:52, FastEthernet0/3
0 172.16.40.0 (110/2) via 10.0.1.1, 00:00:42, FastEthernet0/3
0 172.16.40.0 (110/2) via 10.0.1.1, 00:00:42, FastEthernet0/3
```

- DS SW1 라우팅 테이블(ECMP) -

6. 관리자 PC 연결 (사용자 -> db 접속)



7. DS SW -> ACL 적용 (사용자 -> db 접속)

```
access-list 110 deny ip 192.168.10.0 0.0.0.255 172.16.40.0 0.0.0.255
access-list 110 permit ip any any
```

사용자(vlan10) -> DB(vlan 40) 차단 ACL 적용

```
interface vlan 10
ip access-group 110 in
```

ACL vlan10에 적용

8. R1 & R2 IP 할당 및 OSPF 추가

```
R1(config)#interface g0/0
R1(config-if)# ip address 10.0.3.2 255.255.255.252
R1(config-if)# no shutdown
```

```
R1(config-if)#router ospf 1
R1(config-router)# network 10.0.3.0 0.0.0.3 area 0
```

클라우드 서버에 ping을 위해서 R1, R2 설정

6. 퍼블릭 (클라우드) 네트워크 구축 (테스트용)

기업형 네트워크 구조 적용 (Router-Switch-Server)

```
DS SW1(config)#router ospf 1
DS SW1(config-router)# network 10.0.1.0 0.0.0.3 area 0
DS SW1(config-router)# network 10.0.1.0 0.0.0.3 area 0
```

```
GigabitEthernet0/1
C 10.10.10.0/24 is directly connected, GigabitEthernet0/0
C 10.10.10.1/32 is directly connected, GigabitEthernet0/0
0 172.16.0.0/24 is subnetted, 2 subnets
0 172.16.20.0/24 (110/3) via 200.1.1.1, 00:00:24,
GigabitEthernet0/1
0 172.16.40.0/24 (110/3) via 200.1.1.1, 00:00:24,
```

7. IP SLA 적용

지원하지 않음..

```
ip sla 1
icmp-echo 192.168.40.10 ! 서버 VLAN

ip sla schedule 1 start-time now life forever

track 1 ip sla 1 reachability

ip route 0.0.0.0 0.0.0.0 192.168.10.1 track 1 ! DS + BB1
ip route 0.0.0.0 0.0.0.0 192.168.10.2 track 1 ! DS + BB2
```

[결과1] - ACL 정책 적용 결과

Fire	Last Status	Source	Destination
●	Failed	PC0	DB Server
●	Successful	PC1	DB Server
●	Successful	DB S...	web Server
●	Successful	PC0	Backup S...
●	Successful	PC1	Backup S...

사용자(vlan10) PC -> DB Server

```
C:\>ping 172.16.40.10

Pinging 172.16.40.10 with 32 bytes of data:
Reply from 192.168.10.1: Destination host unreachable.
```

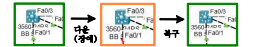
관리자 (vlan99) PC -> DB Server

```
Pinging 172.16.20.10 with 32 bytes of data:
Reply from 172.16.20.10: bytes=32 time=1ms TTL=126
Reply from 172.16.20.10: bytes=32 time=3ms TTL=126
```

웹 서버 -> DB Server

```
Pinging 172.16.20.1 with 32 bytes of data:
Reply from 172.16.20.1: bytes=32 time<1ms TTL=255
Reply from 172.16.20.1: bytes=32 time<1ms TTL=255
```

<기존 이중화 및 경로 우회>



물리적으로 다름이 됐을 때만 경로 우회를 함

IP SLA (IP Service Level Agreement)란?
IP SLA는 네트워크의 지연(Latency)과 패킷 손실(Packet Loss)을



1. 서버방 -> DB 서버 추가 (vlan)

```
SERVER SW(config)#vlan 20
SERVER SW(config-vlan)# name WEB
SERVER SW(config-vlan)#vlan 40
SERVER SW(config-vlan)# name DB
```

2. HSRP 적용 (BB SW1, BB SW2)

Interface	Grp	Pri	State	Active	Standby
Virtual IP					
V20	20	110	Active	local	172.16.20.3
V40	40	110	Active	local	172.16.40.3

HSRP 적용 (BB SW1 Active 장비)

3. web Server & DB Server IP 할당

IPv4 Address	Subnet Mask	Default Gateway	DNS Server
172.16.20.30	255.255.255.0	172.16.20.1	0.0.0.0
IPv4 Address	Subnet Mask	Default Gateway	DNS Server
172.16.40.10	255.255.255.0	172.16.40.1	0.0.0.0

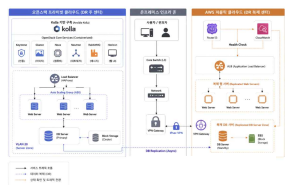
Web Server DB Server

4. Ping 테스트 (게이트웨이)

```
Pinging 172.16.20.1 with 32 bytes of data:
Reply from 172.16.20.1: bytes=32 time<1ms TTL=255
Reply from 172.16.20.1: bytes=32 time<1ms TTL=255
Reply from 172.16.20.1: bytes=32 time<1ms TTL=255
Reply from 172.16.20.1: bytes=32 time<1ms TTL=255
```

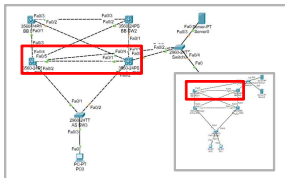
하이브리드 클라우드 기반 온프레미스 인프라 구축 및 운영 구조 설계

1. 장애 대응 기준 수정



- 내부 과부하 및 논리 장애 → OpenStack 내부 자원으로 우회
- 퍼블릭 클라우드 → DR(재해복구) 용도로 사용
- 서버/DB 물리 장애 발생 시 퍼블릭 환경으로 서비스 우회
- 내부 운영 안정성과 외부 재해 복구 역할 분리 구성

2. OpenStack 서버망 위치 구조



- 기존: OpenStack 서버항을 BB SW에 직접 연결
- 수정: VLAN 20(Server Zone) 기반 DS SW 구조로 분리
- 일반 서버와 클라우드 서버를 동일 Server Zone에서 운영
- 논리적 클라우드로 분리 + 물리적 서버별 연결 구조 적용
- Core는 라우팅/스위치 역할에 집중하도록 구성

OpenStack(오픈스택) 이란?

VM 만드는 클라우드 운영체제

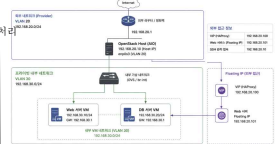
서비스	역할
keystone	인증
Glance	이미지
Nova	VM 관리
Neutron	네트워크
Horizon	웹(UI)
Cinder	블록 스토리지
MariaDB	DB
RabbitMQ	메시지 큐

Kolla-Ansible이란?

OpenStack 자동 설치 관리자.

- Docker 컨테이너를 사용하여 RabbitMQ 예시지
- 오픈스택을 배포하고 관리하기 위한 Ansible 플레이북

3. 프라이빗 클라우드 아키텍처



- OpenStack 기반 프라이빗 클라우드 환경 구성
- 외부망(VLAN20)과 내부 VM망 분리 설계
- OVS 기반 가상 네트워크 및 Floating IP 구성
- Web / DB VM 기반 서비스 구조 설계
- DB 서버 내부망 전용 구성으로 외부 접근 차단
- 관리망(192.168.56.X) 구성으로 운영 안정성 확보

4. 웹 서비스 & DB (추후 변경 있을 수 있음)

웹 서버:
로그인 페이지, 메인 대시보드, 직원/고객 조회 페이지
일반 DB:
직원정보 (이름, 부서, 직급 공지사항예약 내역
보안 DB:
계정/비밀번호 (해시), 고객 개인정보

1) OpenStack 구축 환경 준비



Ubuntu 22.04 기반 VM



2) 네트워크 구성

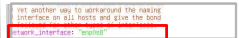
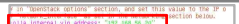
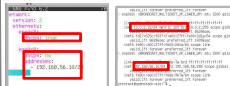


인터넷용 NIC

Openstack 관리용



NIC 활성화된 거 확인 가능 BUT) IP 할당은 아

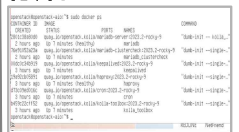


globals.yml 설정



bootstrap-servers& prechecks

[결과1] ^{실행}openstack 설치된 서비스들



OpenStack 기본 인프라 컨테이너는 대부분 올라왔으나 현재 Neutron 네트워크 구성 단계에서 설치가 중단된 상태

[결과2] 웹 서버에 올릴 서비스



OpenStack 기반 사설 클라우드 인프라 구축

1. depoly (배포) 완료

```
PLAY Roleplay role skyline: *****
skipping: no hosts matched

PLAY RECAP *****
localhost: (ok=270) changed=51 unreachable=0 failed=0 skipped=244 rescued=0 ignored=0
```

<http://192.168.56.20> 접속 -> Horizon 대시보드

프라이빗 클라우드 설치 완료!

2. private network (내부망) & 라우터 생성

[illegible]

8. 외부 네트워크 IP 주소 + 게이트웨이 확인

[illegible]

4. External Network (외부망) 설정

```
openstack@openstack-ai0:~$ openstack network create external-net \
  --external \
  --provider-network-type flat \
  --provider-physical-network physnet1
```

external-net 생성

```
--subnet-range 10.0.2.0/24 \
--gateway 10.0.2.1 \
--allocation-pool start=10.0.2.200,end=10.0.2.220 \
--no-dhcp
```

external subnet 생성

5. external network <-> router 연결

```
openstack@openstack-alo:~$  
openstack@openstack-alo:~$ openstack router set router1 --external-gateway external-net
```

router1에 external-net 연결

6. private network <-> router 연결

```
jenstack@openstack-alc:~$ openstack router add subnet router1 private-subnet
jenstack@openstack-alc:~$ openstack router add subnet router1 private-net
```

router1에 private1-net 추가

7. router1 설정 결과

Field	Value
admin_state_up	UP
availability_zones	nova
created_at	2025-05-12T20:04:10Z
description	
discovery_enabled	False
external_gateway_info	{ "external_id": "41465406-6401-b0ef-5f4dc570232c", "external_ip_addresses": [{"subnet_id": "1491965D-0208-4656-ac78-45485e0a0e18", "ip_address": "10.0.2.205"}], "external_network_id": null }
flavor_id	None
ha	False
interfaces_info	{ "port_id": "71b1018e-0459-493c-9759-3c3d970a6b67", "ip_addresses": [{"subnet_id": "70c7794d-4850-447d-4f14-0765f710c411"}] }
name	70c7794d
project_id	4493332c-6341-b46a-75db61e401852
revision_number	3
update	ACTIVE
tags	
updated_at	2025-05-14T08:21:30Z

```
private network <-> router <->
```

ID	Name	Parent ID	Status	Type	Expires Date	Actions
1	Child A (10%)	1 (10.00%)	Done	01/10/2023	1P	Details
2	Child B (5%)	1 (5.00%)	Done	01/10/2023	1P	Details

8. Floating IP 생성

Field	Value
created_at	2026-05-14T18:38:53Z
description	
dns_domain	None
dns_name	None
floating_ip_address	None
floating_ip_address	10.0.2.17
floating_ip_network_id	ef30f544-baac-4489-9152-22c5a55a6c91
name	0.0.2.17
port_details	
port_id	None
project_id	43f33324cd5c947a5a675deb61e49b352

9. ISO 설치 -이미지 생성 (ubuntu-22.04)

jammy-server-cloudimg-amd64 → 우분투 디렉토리로 복사 후 진행

```
penstock@penstock-ali:~$ ls
--file jekyll-server-cloudimg-amd64.img
penstock@penstock-ali:~$ penstock image create "ubuntu-22.04" \
--file jekyll-server-cloudimg-amd64.img \
--disk-format qcow2 \
--container-format bare \
--public \
--property architectures=x86_64 \
--min-disk 20 \
--min-ram 2048
```

OpenStack 안에서 VM 템플릿(OS 원본)을

10. 보안그룹 생성 (web-ag, db-ag)

Web-sg 보안그룹 생성				
☐ 웹-sg	c4aacc18b-2769-470e-9029-11b85f4c4d			
☐ 관리자	IPv4	ICMP	전체	0.0.0.0/0
☐ 관리자	IPv4	TCP	22 (SSH)	0.0.0.0/0
☐ 관리자	IPv4	TCP	80 (HTTP)	0.0.0.0/0
☐ 관리자	IPv4	TCP	443 (HTTPS)	0.0.0.0/0

SSH(22), HTTP(80), HTTPS(443), ICMP 포트 허용

☐ db-sg	704c9fc3-d0ee-4925-9360-28a883336de6		
db-sg 보안그룹 생성			
☐ 들어옴	IPv4	TCP	22 (SSH)
☐ 들어옴	IPv4	TCP	3306 (MySQL)

SSH(22), MySQL(3306) 포트 허용

10. flavor 선행

ID	Name	Risk	Disk	Ephemeral	ICPIs	Is Public
ba082f26-266d-4aa0-a592-9f3c6a2ba560	web-smail	S12	5	0	1	True
cc98a920-7624-40b0-ba77-84c3a9fa421f	db-medium	S12	5	0	1	True

이미지(ISO와 크기가 다르면 안됨.

최소 디스크	5
최소 RAM	512

11. 키 페어 생성

☐ 0월 *	0월
☐ 1월	1월
☐ 2월	2월

12. VM 생성 (web-server, db-server)

```
openstack@openstack-ai:~$ openstack server create
> --flavor web-small \
> --image 94b47ad5-d554-4cac-8903-0dda1ef9acd2 \
> --network private-net \
> --security-group web-sg \
> --key-name web-key \
> web-server
```

```
penstack@openstack-alo:~$ openstack server create \
--flavor db-medium \
--image 94b47ad5-d554-4cac-8903-0dda1ef9acd2 \
--network private-net \
--security-group db-sg \
--key-mnae db-sg \
--key-name db-key \
db-server
```

[결과] -경로포임

Instance Name	Image Name	IP Address	Flavor	Key Pair	Status
db-server	ubuntu-22.04		db-medium	db-key	Error
web-server	ubuntu-22.04		web-small	web-key	Error

ID	Name	Status	Networks	Dnsge	Flavor
5317c9a6-b188-4e9b-b07e-636760bf1017	db-server	ERROR		ubuntu-22.04	db-medium
af194441-a170-4c72-4c3c-ba75a259fb68	web-server	ERROR		ubuntu-22.04	web-small

OpenStack VM 생성 및 네트워크 트러블슈팅 & 인터넷 연결

[실험]

문제
1. 네트워크 연결
2. KVM 지원 x

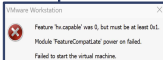
오류: 인스턴스 "web-server"에 대해 요청된 동작 수행 실패, 인스턴스에 여러 상태가 있습니다. 다음에 다시 시도하십시오 [오류: Exceeded maximum number of retries. Exhausted all hosts available for retrying build failures for instance a1ff18743-...

VIII 생성 X

```
openstack@openstack-iso:~$ sudo docker ps | grep openstack
scalex/mri quay.io/openstack/kilo:a9e0c76-swiftch/2023-2-ricky-9 "dumb-init -s --sig
          5 minutes ago 0 seconds (healthy) a9e0c76/swiftch
00c83b75545 quay.io/openstack/kilo:a9e0c76-db-server/2023-2-ricky-9 "dumb-init -s --sig
          5 minutes ago 0 3 minutes (healthy)
00c20f98bd0 quay.io/openstack/kilo:neutron-openstack-agent/2023-2-ricky-9 "dumb-init -s --sig
          26 hours ago 0 44 seconds \(healthy\) neutron\_a9e0c76\_agent
openstack@openstack-iso:~\$
```

네트워크 서비스 삭제 후 재설치

```
openstack@openstack-ai0:~$ ls /dev/kvm
ls: cannot access '/dev/kvm': No such file or directory
```



kvm 지원 확률 높은 VMware로 옮겼지만 지원 X

```
jenstack@jenstack-aj:~$ sudo grep virt_type
virt_type = kvm
```

KVM 제거 QEMU로 변경

ID	Name	Status	Networks	Image	Flavor
4c49b114-6301-4415-a01e-5132ba3d3eb3	db-server	ACTIVE	private 172.16.10.37	ubuntu-22.04	db.sml
74e99028-2182-4859-8039-36ada5318c47	web-server	ACTIVE	private 172.16.10.112	ubuntu-22.04	web.sml

OpenStack 내부·외부망 통신 검증

[illegible]

내부망 ping성공

외부땅 ping성공

OpenStack 외부 접근 및 SSH 연결 (VM 내부 접속)

```
# Connect to Ubuntu 22.04.3 LTS [Linux-6.8.0-27-generic] (en_US)
# Documentation: http://help.ubuntu.com
# Management:    http://www.ubuntu.com/LTS
# Support:        https://discuss.ubuntu.com/txt

System Information as at Tue May 18 09:30:30 UTC 2020

System load:   0.00      Processes:           90
Usage of / :   8.5% of 31.2GB            Swap:              0B/0B
Memory usage : 10k       IP address(es):     192.168.1.100
Loop size:     1MKB      IPsec status:        off
```

```

The list of available updates is more than a week old.
To check for new updates run: sudo apt update

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described
in individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY. It is an exact price
applicable law.

To view a complete and unabbreviated license (GPL), run 'dpkg-query

```

외부 Windows 환경에서 Floating IP를 통해 OpenStack 내부 VM(Web Server)에 SSH 접속 성공

```

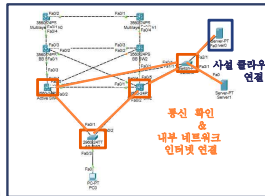
VM#
172.16.10.1(grouter)
SNAT
외부 인터넷
ver:~$ ping 8.8.8.8
8.8.8.8: 56(84) bytes of data.
201 icmp_seq=1 Destination Host Unreachable
201 icmp_seq=2 Destination Host Unreachable
201 icmp_seq=3 Destination Host Unreachable
201 icmp_seq=4 Destination Host Unreachable

```

vm -> 외부 통신 불가

```
ubuntu@ubuntu-server:~$ ping 10.15.70.67
PING 10.15.70.67 (10.15.70.67) 56(84) bytes of data:
From 10.15.70.201 icmp_seq=1 Destination Host Unreachable
From 10.15.70.201 icmp_seq=2 Destination Host Unreachable
From 10.15.70.201 icmp_seq=3 Destination Host Unreachable
```

내부망 게이트웨이까지는 전선이 가는데 외부망쪽은 V



통신 확인 및 라우팅 수정 (ECMP -> Floating)

The diagram illustrates the transformation of a 3D point cloud into a 2D feature map. On the left, a 3D point cloud is shown with coordinates (x, y, z) and a color value c . The point cloud is represented as a set of points $\{(x_i, y_i, z_i, c_i)\}_{i=1}^N$. The right side shows a 2D feature map with dimensions (x, y) and a color value c . The feature map is represented as a set of points $\{(x_i, y_i, c_i)\}_{i=1}^N$. A large arrow points from the 3D point cloud to the 2D feature map, indicating the transformation process.

두 경로 중 고민하는 시간 길어짐 flating^오

인터넷 연결 (R4, R5) 라우팅 추가

```
Enter configuration commands, one per line. End with CNTL/Z.
R4(config)#ip route 192.168.10.0 255.255.255.0 10.1.1.14
R4(config)#ip route 192.168.10.0 255.255.255.0 10.1.1.13
R4(config)#ip route 192.168.20.0 255.255.255.0 10.1.1.14
R4(config)#ip route 192.168.20.0 255.255.255.0 10.1.1.13
R4(config)#ip route 192.168.30.0 255.255.255.0 10.1.1.14
R4(config)#ip route 192.168.30.0 255.255.255.0 10.1.1.13
R4(config)#
```

목적지 vlan 10 ~ vlan 30 추가

[illegible]

vlan20과 vlan30 인터넷 연결 성공 vlan 10

```
Success rate is 100 percent (5/5), round-trip min/avg/max = 1/3/9 ms
0x00-000000 0.0.0.0 0.0.0.0 0.0.0.0

Type escape sequence to abort.
Sending 5, 100-byte ICMP Echoes (5 packets) to 192.168.1.254:
Packet sent with a source address of 192.168.1.254
Success rate is 0 percent (0/5)
0x00-000000 0.0.0.0 0.0.0.0 0.0.0.0
Switch-2#
```

외부망 IP 대역과 IP 충돌 valn 10 IP

[illegible]

SW2, SW6 연결 성공



IP 주소:	수동
IPv4 주소:	192.168.20.100
IPv4 마스크:	255.255.255.0
IPv4 게이트웨이:	192.168.20.254
DNFS 서버 설정:	수동
IPv4 DNS 서버:	134.27.217.217 (공공 DNS) 없음 168.252.85.252 (공공 DNS) 없음

[illegible]

WireGuard VPN 연결 및 DNS 전환 설계

```
if packages can be upgraded: run: apt list --upgradable to see them
yboxuser@suricata:~$ sudo apt install -y wireguard
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
  wireguard-tools
```

와이어 가드 설치

```
yboxuser@suricata:~$ wg genkey | tee privatekey | wg pubkey > publickey
yboxuser@suricata:~$ cat privatekey
GJ//skUEaHjR7JeATAbAavdL3usjxBSUL/9Z106yPH4=
yboxuser@suricata:~$ cat publickey
qf4QjB5udhwCEj6cvE1WkkrminIdyMowCtLIUIYHFA=
yboxuser@suricata:~$
```

퍼블릭 / 개인 키 생성

```
GNU nano 2.9.3 /etc/wireguard/wg0.conf
[Interface]
PrivateKey = GJ//skUEaHjR7JeATAbAavdL3usjxBSUL/9Z106yPH4=
Address = 10.200.0.2/24

[Peer]
PublicKey = 2KvYbU/AZ9FAW0aGtGtE4RGd/dHwMh77kys=
Endpoint = 54.180.118.141:51820
AllowedIPs = 10.0.0.0/16
PersistentKeepalive = 25
```

설정 창 (EC2)

```
yboxuser@suricata:~$ sudo wg-quick up wg0
[#] ip link add wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.200.0.2/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
[#] ip -4 route add 10.0.0.0/16 dev wg0
```

```
GNU nano 2.9.1 /etc/wireguard/wg0.conf
[Interface]
PrivateKey = +0CKUEIRqWREK9EKGmVfmoD9rjC8CjE51fain=
Address = 10.200.0.1/24
ListenPort = 51820

[Peer]
PublicKey = qf4QjB5udhwCEj6cvE1WkkrminIdyMowCtLIUIYHFA=
AllowedIPs = 192.168.0.0/16
```

설정 창 (방화벽)

```
sudo systemctl enable wg-quick@wg0
```

인터페이스 생성 & 재시작 시 자동실행

```
yboxuser@suricata:~$ ping 10.200.0.1
PING 10.200.0.1 (10.200.0.1) 56(84) bytes of data:
64 bytes from 10.200.0.1: icmp_seq=1 ttl=64 time=11.0 ms
64 bytes from 10.200.0.1: icmp_seq=2 ttl=64 time=11.5 ms
64 bytes from 10.200.0.1: icmp_seq=3 ttl=64 time=10.1 ms
64 bytes from 10.200.0.1: icmp_seq=4 ttl=64 time=10.7 ms
64 bytes from 10.200.0.1: icmp_seq=5 ttl=64 time=11.0 ms
64 bytes from 10.200.0.1: icmp_seq=6 ttl=64 time=11.3 ms
^C
--- 10.200.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5013ms
rtt min/avg/max/mdev = 10.086/10.933/11.456/0.436 ms
yboxuser@suricata:~$
```

ping 통신 가능

```
ubuntu@ip-172-31-32-43:~$ ping 10.200.0.2
PING 10.200.0.2 (10.200.0.2) 56(84) bytes of data:
64 bytes from 10.200.0.2: icmp_seq=1 ttl=64 time=10.0 ms
64 bytes from 10.200.0.2: icmp_seq=2 ttl=64 time=10.0 ms
64 bytes from 10.200.0.2: icmp_seq=3 ttl=64 time=11.3 ms
64 bytes from 10.200.0.2: icmp_seq=4 ttl=64 time=10.2 ms
^C
--- 10.200.0.2 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 9.995/10.379/11.266/0.518 ms
```

장애 감지 -> 자동 재시작 -> DNS Failover

[전체 흐름]

```
crontab으로 5초마다 자동 실행
|
DS SW ping / Wiki.js HTTP / PostgreSQL TCP 체크
장애 감지 시 재시작 1회 시도
|
실패 시 즉시 AWS DR 전환
|
온프레미스 복구 감지 시 자동 원복
```

[장애 판단 기준]

DS SW1 + SW2 둘 다 실패 -> DR 전환
Wiki.js HTTP 실패 -> 재시작 -> 실패시 DR
PostgreSQL TCP 실패 -> 재시작 -> 실패시 DR
DS SW 단독 장애 -> 반대편으로 트래픽 전환

[스크립트 구성]

check_onprem_status.sh -> 장애 감지 + 판단
change_dns_to_aws.sh -> DNS -> AWS 전환
change_dns_to_onprem.sh -> DNS -> 온프레미스 원복

정상 운영 중 모니터링 로그

```
ubuntu@monitor-server:~$ tail -f /var/log/dr_check.log
2026-06-07 16:53:01 INFO 온프레미스 정상 운영 중
2026-06-07 16:53:30 INFO 온프레미스 정상 운영 중
2026-06-07 16:53:31 INFO 온프레미스 정상 운영 중
2026-06-07 16:53:31 INFO 온프레미스 정상 운영 중
2026-06-07 16:54:01 INFO 온프레미스 정상 운영 중
2026-06-07 16:54:01 INFO 온프레미스 정상 운영 중
2026-06-07 16:54:31 INFO 온프레미스 정상 운영 중
```

Wiki.js 장애 + DR 전환 로그

```
ubuntu@monitor-server:~$ tail -f /var/log/dr_check.log
2026-06-07 13:14:49 INFO Wiki.js Docker 재시작 시도
2026-06-07 13:14:49 WARN 연속 실패 1 / 3
2026-06-07 13:15:51 INFO Wiki.js HTTP 실패 (http://192.168.20.20)
2026-06-07 13:15:51 INFO Wiki.js Docker 재시작 시도
2026-06-07 13:15:51 WARN 연속 실패 2 / 3
2026-06-07 13:16:52 INFO Wiki.js HTTP 실패 (http://192.168.20.20)
2026-06-07 13:16:52 INFO Wiki.js Docker 재시작 시도
2026-06-07 13:16:52 INFO 연속 실패 3 / 3
2026-06-07 13:16:52 CRITICAL 임계 작전 - AWS DR 전환
2026-06-07 13:17:35 DR DNS 전환 완료: service.local + 172.31.32.43
```

nslookup service.local

```
서버: Unknown
Address: 192.168.56.10

이름: service.local
Address: 192.168.20.20
```

```
서버: Unknown
Address: 192.168.56.10

이름: service.local
Address: 172.31.32.43
```

복구 후 로그

```
ubuntu@monitoring:~$ tail -f /var/log/dr_check.log
RECOVERY DNS 복구 완료: service.local + 192.168.20.20
RECOVERY DNS 복구 완료: service.local + 192.168.20.20
INFO 온프레미스 정상 운영 중
INFO 온프레미스 정상 운영 중
INFO 온프레미스 정상 운영 중
INFO 온프레미스 정상 운영 중
INFO 온프레미스 정상 운영 중
```


실험 결과 (DR 모니터링)

[모니터링 구성]

Prometheus + Blackbox

-5초마다 ping/HTTP/TCP 체크

Node.js /logs API (포트 3001)

-logs -> 로그 /dns -> dns 상태

React 대시보드 (포트 5173)

실시간 시각화 + RTO 계산

[데이터 흐름]

Prometheus -> 노드 색상

Node.js /logs -> 이벤트 타임라인

Node.js /dns -> route-pill / RTO 타이머 / AWS DR 상태

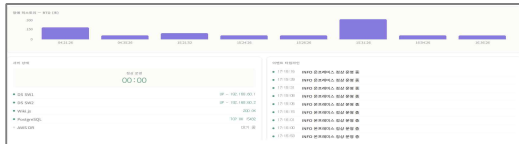
[Prometheus 수집 항목]

ds sw1: 192.168.60.1 ICMP(ping)

ds sw2: 192.168.60.2 ICMP(ping)

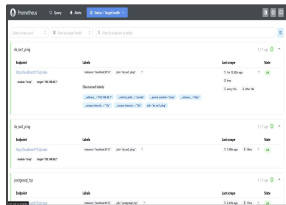
wikijs: 192.168.20.20 HTTP(200)

**postgrasql: 192.168.30.30:5432
TCP connect**



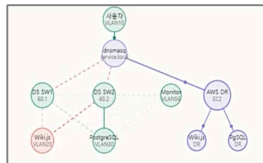
<http://192.168.56.10:5673> 접속

-Prometheus 연동 상태-



정상적으로 동작 중

- 노드에 장애가 발생했을 때-



노드 빨간색 + AWS DR 운영으로 바뀜